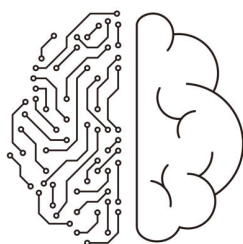




UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

Deep Learning: BERT and DistilBERT in NLP



Student name: Antonis Zikas

email: sdi2100038@di.uoa.gr

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Spring Semester 2025*

Contents

1	Data Processing and Analysis	2
1.1	Data Pre-processing	2
1.1.1	General Cleaning	2
1.1.2	Stop-words Removal	2
1.2	Exploratory Data Analysis	3
1.2.1	Sentiment Correlation	3
1.2.2	Dataset Splitting	3
1.2.3	Sentiment Distribution	3
1.2.4	Analysis of Word Frequencies by Sentiment	4
1.2.5	Text Length Analysis	4
2	Tokens and Experimentation	5
2.1	BERT and DistilBERT Tokenization	5
2.2	Experimentation	5
2.2.1	Dummy Model	5
2.2.2	Base Model	5
2.2.3	Advanced Model	6
2.2.4	Layer Freezing	6
2.2.5	Hyperparameter Tuning	7
3	Results and Overall Analysis	8
3.1	Experimental Summary	8
3.2	Overall Comparison and Limitations	8
3.3	Conclusion	8
4	Bibliography	8

ABSTRACT

In this article, we explore the field of **Natural Language Processing** (NLP) by developing a sentiment analysis system that classifies user tweets as either *positive* or *negative*. To achieve this, we leverage the power of **pretrained transformer models**, specifically **BERT** [1][2] and its lighter variant **DistilBERT** [1][3]. We fine-tune both models on our dataset and evaluate their performance to determine the effectiveness of transfer learning in this binary sentiment classification task.

KEYWORDS. Machine Learning, Sentiment Classifier, BERT, DistilBERT

CODE LINKS. Kaggle notebooks: [BERT](#) and [DistilBERT](#)

1 Data Processing and Analysis

The dataset is first **processed** and **cleaned**. In data analysis and machine learning, data preprocessing is considered the most critical initial phase of the workflow. Ensuring that the data is in a **suitable format** is essential for effective algorithmic applications.

1.1 Data Pre-processing

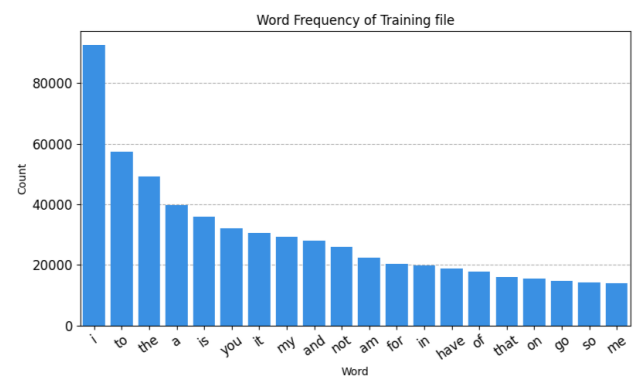
1.1.1. General Cleaning. The dataset underwent several preprocessing and cleaning techniques, including: 1) **Removal of URLs, user mentions, hashtags, special characters, and numbers**, 2) **Lowercasing all characters**, 3) **Elimination of extra whitespaces**, 4) **Stemming and lemmatization** of words, and 5) **Removal of punctuation**.

Applying these methods to the dataset yielded the results shown in Table 1, which displays a representative subset of transformed tweets. The table demonstrates how the preprocessing steps modified the text: user mentions were removed, characters were converted to lowercase, and other changes were applied.

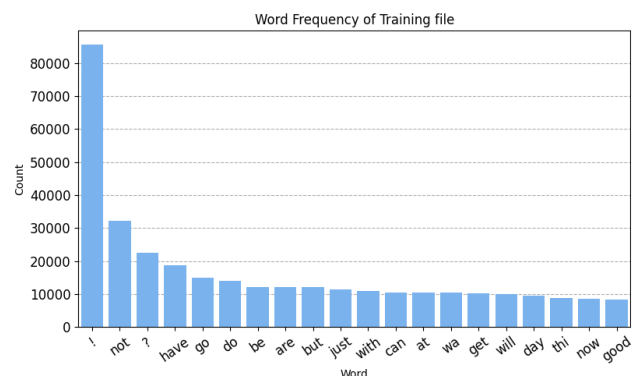
Notably, certain punctuation marks—**exclamation points** (!), **question marks** (?), and **ellipses** (...)—were intentionally retained due to their potential sentiment value. For instance, an **exclamation point** (!) may indicate enthusiasm, while **ellipses** (...) might suggest hesitation or irony.

1.1.2. Stop-words Removal. The subsequent preprocessing phase focuses on eliminating words with minimal sentiment value, known as **stop-words**. These terms provide negligible benefit for model training due to their limited semantic content. As such words frequently dominate text corpora without meaningful interpretation, their removal was prioritized.

A custom stop-word removal approach was implemented, differing from standard library-based methods. The processed dataset results are shown in Table 1, with the complete preprocessing workflow visualized in Figure 1 for the **training file**.



(a) General Cleaning word frequency



(b) Stop-words removal word frequency

Figure 1: Most frequent words across preprocessing steps

ID	Original Text	General Cleaning	Stop-words Removal
39015	how much i do love him	how much i do love him	how much do love him
15358	@Simpsdj Cool stuffs, sounds interesting	cool stuff sound interest	cool stuff sound interest
7666	@zackalltimelow dude what is it like	dude what is it like	dude what like
103061	on my way to METROBAR... teleserv anniv	on my way to metrobar... teleserv anniv	way metrobar... teleserv anniv

Table 1: Text Transformation after preprocessing. Second column shows the original text, the third column represents the text after general cleaning and fourth column shows the text after removing stop-words.

1.2 Exploratory Data Analysis

We will now proceed to some deeper analysis of the dataset.

1.2.1. Sentiment Correlation. The Exploratory Data Analysis (EDA) commenced with **sentiment correlation analysis** to identify words most frequently associated with each sentiment. The dataset was divided into positive and negative **subsets**, followed by extraction of frequent terms. The results are presented in Figure 2.

1.2.2. Dataset Splitting. In machine learning applications, proper dataset partitioning is essential for model development. The standard approach divides data into three distinct subsets: **70% for training**, **20% for validation**, and **10% for testing**. This structured division facilitates effective model training, hyperparameter tuning, and final evaluation. The current dataset's file size distribution across these splits is presented in Table 2, with additional visualization provided in Figure 3.

Dataset File	Records Count	Size Rate
Training	593.552	$\approx 70.00\%$
Validating	169.584	$\approx 20.00\%$
Testing	63.597	$\approx 10.00\%$

Table 2: Dataset training, validating and testing files size

1.2.3. Sentiment Distribution. A **Sentiment Distribution Analysis** was conducted for both training and validation datasets. This analysis quantified the proportion of positive versus negative comments in each dataset. The results, presented in Table 3 and Figure 3, reveal near-equal representation of both sentiment classes. While **positive comments** slightly **outnumber** negative ones in both datasets, the marginal difference confirms their **balanced** distribution. This equilibrium suggests neither sentiment class dominates the training or validation data.

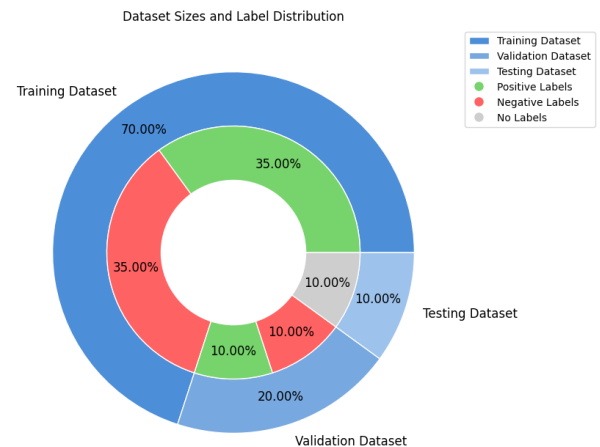


Figure 3: Training, Validating and Testing Datasets sizing and sentiment distribution analysis

Data Balance. The balanced dataset ensures a **fair learning process**, where the model is not biased toward any particular class due to unequal representation. This equilibrium promotes better generalization across all classes and yields more reliable validation performance. Consequently, evaluation metrics including **accuracy**, **precision**, **recall**, and **F1-score** provide meaningful assessments of the model's predictive capabilities.

Dataset File	Positive Tweets	Negative Tweets
Training	74.196	74.192
Validating	21.199	21.197
Testing	-	-

Table 3: Sentiment Distribution across the files. How many positive and negative tweets are there.

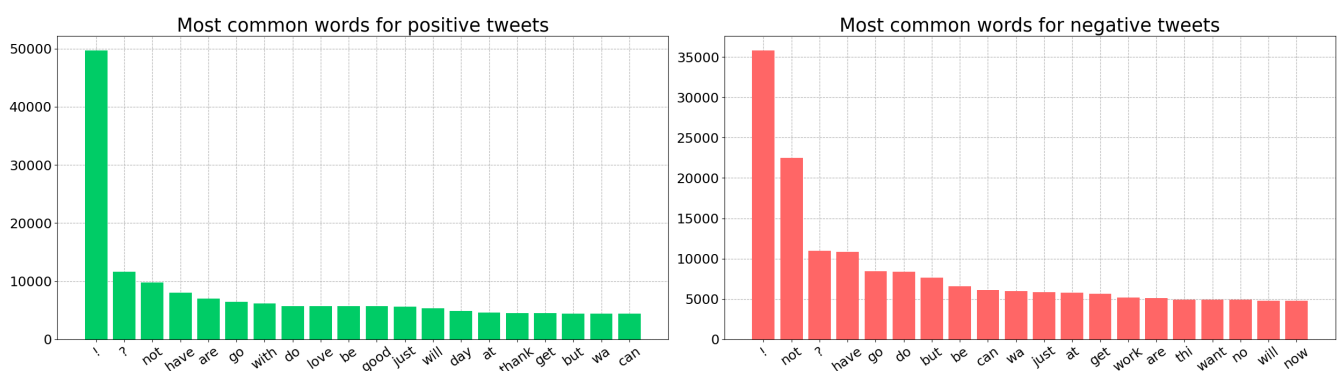


Figure 2: Sentiment Correlation: Most frequent words for positive and negative tweets file

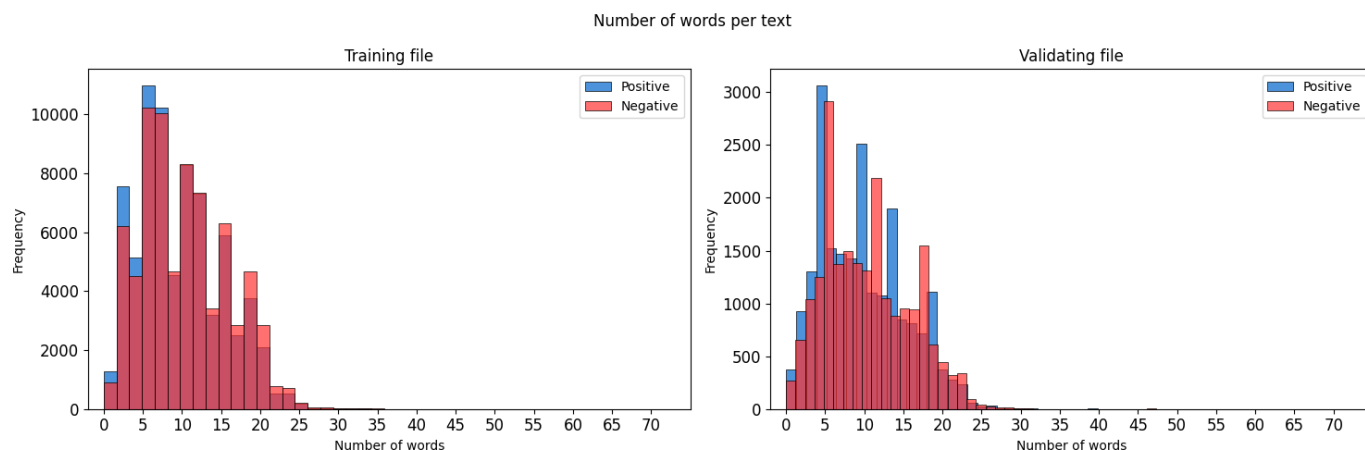


Figure 4: Sentiment Text Length Analysis for both training and validation files. X-axis represents the number of words and y-axis is the frequency of these numbers inside the files.

1.2.4. Analysis of Word Frequencies by Sentiment. Let's take a closer look at which words appear most often in our dataset and how they differ between positive and negative tweets. Figure 1 gives us an overview of the most common words overall, while Figure 5 specifically compares how these top 20 words appear in positive versus negative tweets.

Some interesting patterns stand out when we examine these results. Positive tweets tend to use certain features much more frequently - for example, **exclamation marks (!)** appear more often in positive tweets compared to negative ones. On the other hand, words that express negation like **'not'** show up significantly more in negative tweets. Other words like "good" and "great" naturally appear more in positive tweets, while terms like "bad" and "worst" dominate in negative ones. These patterns make perfect sense when we think about how people express positive versus negative feelings in their writing. The clear differences in word usage help our model learn to distinguish between these two sentiment categories.

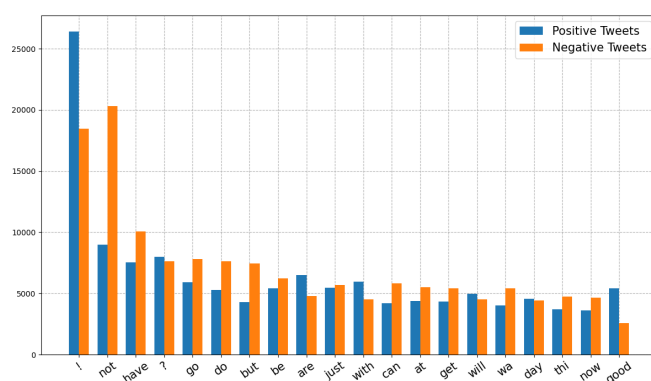


Figure 5: Comparative Analysis of Word Frequencies by Sentiment. How often does a word appear in positive and negative tweets.

1.2.5. Text Length Analysis. Now let's examine how long the tweets are in our dataset. We analyzed the number of words in both our **training set** (which contains 70% of our data) and our **validation set** (containing 20%). Looking at Figure 6, we can see that most tweets are quite short, with the majority falling between 0 to 30 words.

In the training data, the most common tweet lengths are 5-6 words, while in the validation set it's slightly shorter at 4-5 words. The validation set shows smaller numbers overall, but this is exactly what we'd expect since it's a smaller sample of data. When we separate tweets by sentiment (shown in Figure 4), we notice that positive and negative tweets have almost identical length distributions. Whether someone is expressing something positive or negative doesn't seem to affect how long their tweet is - both types average around the same word count. This tells us that text length probably won't be a useful feature for our sentiment classifier, since it doesn't correlate with sentiment.

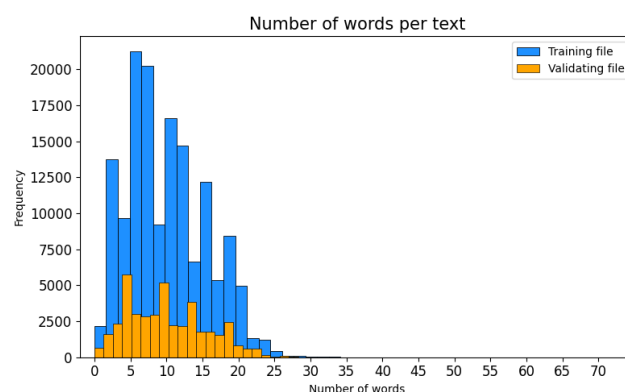


Figure 6: Text Length Analysis for training and validation dataset files. X-axis represents the number of words and y-axis is the frequency of these numbers inside the files.

2 Tokens and Experimentation

2.1 BERT and DistilBERT Tokenization

In this section, we explore the two transformer models used in our experiments: **BERT** and **DistilBERT**. Specifically, we utilized the pretrained versions `bert-base-uncased` and `distilbert-base-uncased` available through the HuggingFace library.

To properly tokenize the tweet data, we first determined the maximum length of the tweets, which is an important parameter for both tokenization and model input formatting. The maximum sequence lengths obtained using the respective tokenizers were **144** for BERT and **161** for DistilBERT.

Due to the large size of the dataset and the computational complexity of the BERT architecture, we used **only 60%** of the data for training and experimentation to ensure efficiency and faster training times.

During tokenization, we included essential special tokens such as [CLS], [SEP], and [PAD] to ensure compatibility with the models. Additionally, we extracted **attention masks**, which are crucial for helping the models distinguish between actual tokens and padded input, and play a significant role during the training process.

2.2 Experimentation

2.2.1. Dummy Model. As with the previous studies, our first experiment serves as a baseline by using a model that makes **random predictions** about the sentiment of each tweet. We refer to this version as the **dummy model**, since it does not involve any actual training or learning process.

The expected outcome of this experiment is a validation accuracy close to **50%**, given the binary nature of the classification task. This model acts as a reference point to evaluate the effectiveness of our fine-tuned transformer models.

The detailed validation metrics for the dummy model are presented in Table 4.

Model	Accuracy	Precision	Recall	F1-score
BERT	0.4984	0.4986	0.4979	0.4983
DistilBERT	0.4996	0.4998	0.4950	0.4974

Table 4: Validation metrics for the Dummy Model with random sentiment predictions.

We also extracted the confusion matrices for both versions of the BERT models, as shown in Figure 7. As expected, the results from the dummy classifier demonstrate poor performance, with predictions nearly evenly split between positive and negative classes. This validates our baseline assumption that the dummy model provides no meaningful predictive power. These results highlight the necessity of incorporating a proper fine-tuned model in order to significantly improve performance.

2.2.2. Base Model. After evaluating an untrained (dummy) model, it is now time to focus on models that undergo actual training. In this section, we introduce the **base model**, which will serve as a baseline for future experimentation. For both

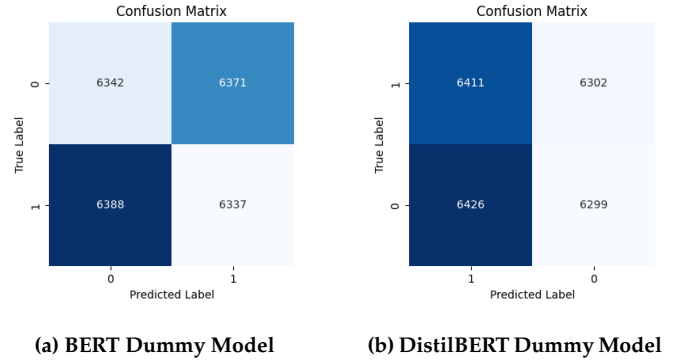


Figure 7: Confusion matrices of the dummy classifiers for BERT (a) and DistilBERT (b). Both models make random predictions and result in near-uniform distributions across prediction classes.

BERT and DistilBERT, we used the `AutoModelForSequenceClassification` class from HuggingFace’s Transformers library, and trained them using the following hyperparameters:

- Number of epochs: 3
- Learning rate (α): **2e-5** = 0.00002
- Epsilon (ϵ): **1e-8** = 0.00000001
- Optimizer: **Adam**

Both models were trained with these settings, and the training and evaluation results are summarized in Figure 8.

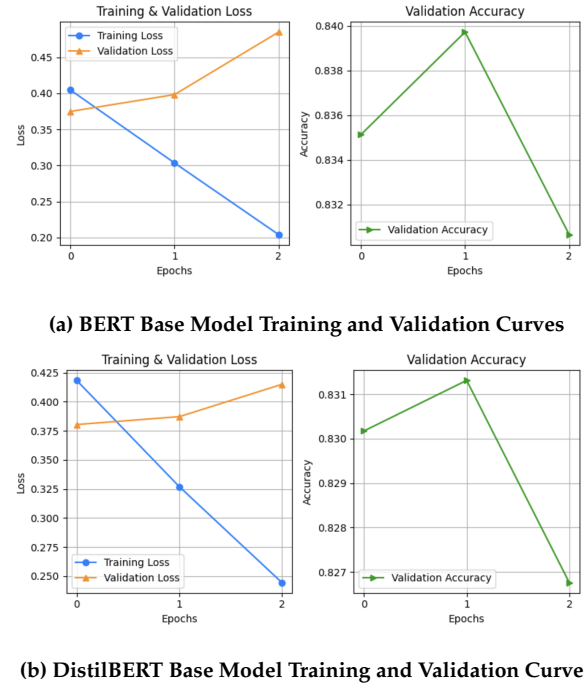


Figure 8: Training and validation performance of BERT and DistilBERT base models using 4 epochs, learning rate = 2e-5, epsilon = 1e-8, and Adam optimizer.

After evaluating the models, the results obtained are shown in Table 5, which includes all the relevant evaluation metrics.

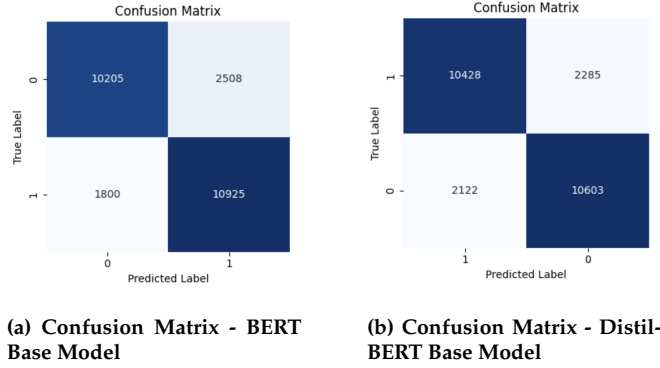


Figure 9: Confusion matrices of BERT and DistilBERT base models after evaluation, clearly improving upon the dummy classifier.

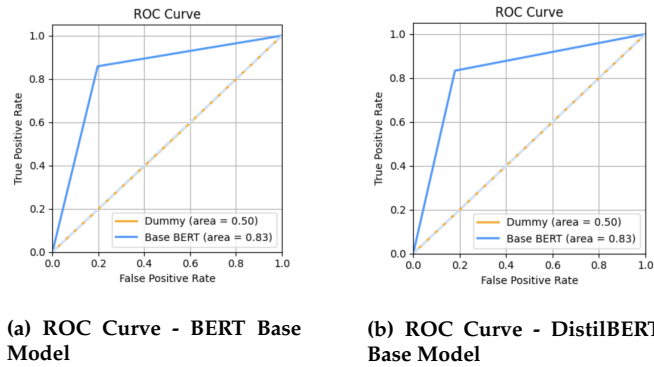


Figure 10: ROC curves of BERT and DistilBERT base models, showing their classification performance on the validation set.

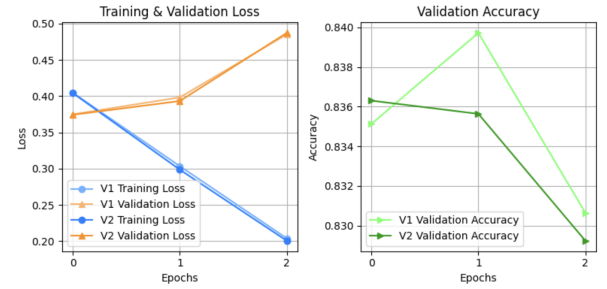
Additionally, Figure 10 presents the ROC curves for the BERT and DistilBERT base models, highlighting their ability to distinguish between classes. Lastly, the updated confusion matrices are visualized in Figure 9. These clearly demonstrate improved classification performance compared to the dummy models.

Model	Accuracy	Precision	Recall	F1-score
BERT	0.8306	0.8132	0.8585	0.8353
DistilBERT	0.8267	0.8227	0.8332	0.8279

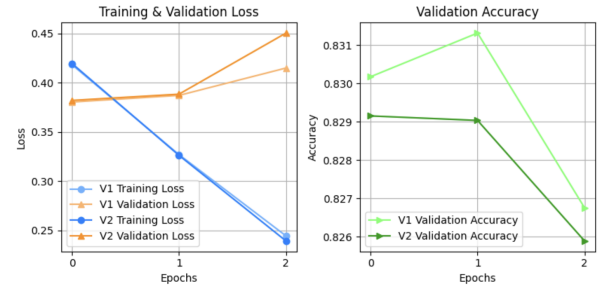
Table 5: Evaluation metrics for the base models of BERT and DistilBERT, using 4 training epochs, a learning rate of $2e-5$, epsilon of $1e-8$, and the Adam optimizer.

2.2.3. Advanced Model. In this section, we experimented with a more advanced version of the models, specifically using BertForSequenceClassification and DistilBertForSequenceClassification individually. The hyperparameters remained the same as in the base model. The results obtained from the training procedure are illustrated in Figure 11.

As we can observe, the learning curves of the training process (training loss and validation loss) behave similarly to those of the base models. The BERT model demonstrates nearly



(a) BERT Advanced Model Training and Validation Curves



(b) DistilBERT Advanced Model Training and Validation Curves

Figure 11: Training and validation performance of BERT and DistilBERT advanced models using 4 epochs, learning rate = $2e-5$, epsilon = $1e-8$, and Adam optimizer.

identical behavior to its base version, whereas the DistilBERT variant shows a slight overfitting effect after the second epoch. Additionally, we notice that the accuracy in both cases is marginally lower compared to the base models.

Taking this into account, we could initially conclude that this version performs slightly worse. However, after conducting multiple additional experiments (not visualized here), we found that the advanced model version consistently outperformed the base model across most runs. Therefore, we will proceed with this advanced model configuration in the subsequent sections.

We evaluated the performance of these advanced models, and the results are presented in Table 6.

Model	Accuracy	Precision	Recall	F1-score
BERT	0.8292	0.8135	0.8544	0.8334
DistilBERT	0.8258	0.8081	0.8548	0.8308

Table 6: Evaluation metrics for the base models of BERT and DistilBERT, using 4 training epochs, a learning rate of $2e-5$, epsilon of $1e-8$, and the Adam optimizer.

2.2.4. Layer Freezing. An interesting feature of the BERT architecture is its flexibility, allowing us to modify and adapt it to our specific needs. In this section, we experimented with **layer freezing**, specifically freezing the first 6 layers of both BERT and DistilBERT before training. The results from this experiment are shown in Figure 12.

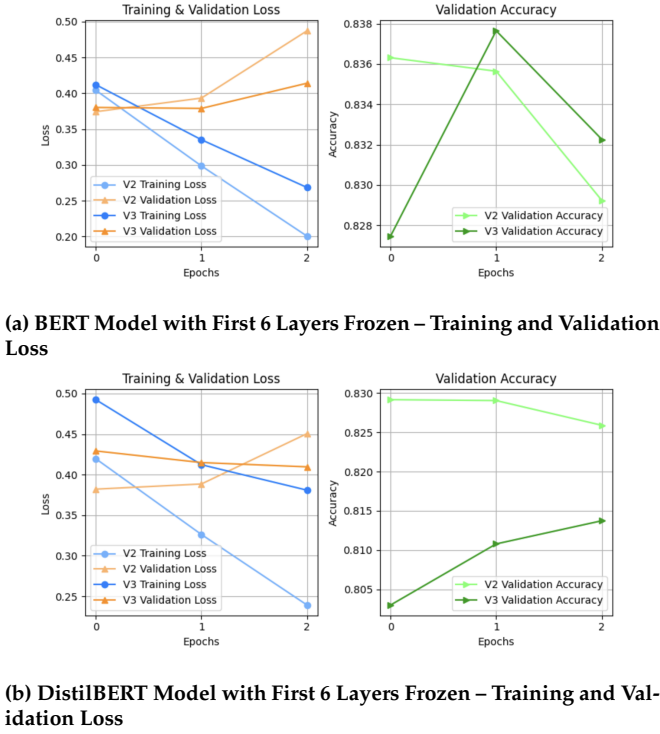


Figure 12: Training and validation performance of BERT and DistilBERT models after freezing the first 6 layers. Training used 4 epochs, a learning rate of $2e-5$, $\epsilon = 1e-8$, and the Adam optimizer.

From the training curves, we observe that the models exhibit improved convergence and reduced overfitting, as indicated by the closer alignment of training and validation losses. While the accuracy of the BERT model slightly improved under this configuration, the DistilBERT model showed a noticeable drop in performance. This discrepancy was surprising, and after conducting multiple experiments, we consistently found that freezing layers generally led to degraded performance—especially for DistilBERT.

Based on these observations, we decided to discard the layer-freezing approach and continue using the full model without freezing any layers. The evaluation metrics on the validation sets for both models are presented in Table 7 and visualized in Figure 13, which includes the confusion matrices and ROC curves for all the model versions so far.

Model	Accuracy	Precision	Recall	F1-score
BERT	0.8322	0.8435	0.8159	0.8295
DistilBERT	0.8137	0.8188	0.8059	0.8123

Table 7: Evaluation metrics for BERT and DistilBERT after freezing the first 6 layers. Training was conducted with 4 epochs, a learning rate of $2e-5$, $\epsilon = 1e-8$, and the Adam optimizer.

2.2.5. Hyperparameter Tuning. Up to this point, we have experimented with different architectural variations of the BERT and DistilBERT models. In this section, we focus on fine-tuning

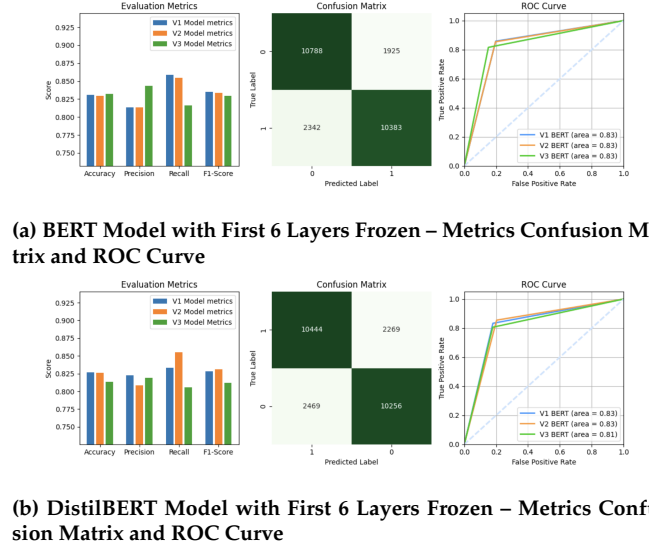


Figure 13: Evaluation results of BERT and DistilBERT models after freezing the first 6 layers, showing evaluation metrics, confusion matrices and ROC curves. Models were trained using 4 epochs, learning rate = $2e-5$, $\epsilon = 1e-8$, and the Adam optimizer. Featuring all versions of the model until now

the training process by optimizing the hyperparameters using the **Optuna** framework. Given the large scale of our dataset—even after using only 60% of it—it is computationally infeasible to exhaustively search through all possible combinations. Therefore, we used Optuna to automate the optimization process and find high-performing hyperparameter configurations efficiently.

We conducted a study with 5 trials, targeting the following hyperparameters:

- **Learning Rate** (α) $\in \{1e-5, 2e-5, 3e-5\}$
- **Epsilon** (ϵ) $\in \{1e-6, 1e-7, 1e-8\}$
- **Weight Decay** $\in \{0.0, 0.01, 0.001\}$
- **Batch Size** $\in \{16, 32\}$
- **Epochs** $\in \{2, 3\}$

The best-performing configurations, as determined by validation accuracy, are summarized in Table 8. The corresponding training curves for both models are visualized in Figure 14 and Figure 15. The best accuracy achieved was **0.8324** for BERT and **0.8241** for DistilBERT.

Model	LR (α)	Eps (ϵ)	WD	BS	Epochs
BERT	$3e-5$	$1e-8$	0.0	16	2
DistilBERT	$3e-5$	$1e-7$	0.0	16	3

Table 8: Best hyperparameter combinations found via Optuna for BERT and DistilBERT. The study was optimized for 5 trials using validation accuracy as the objective metric.

Experiment	Model	Acc.	Prec.	Rec.	F1
Dummy Model	BERT	0.498	0.499	0.498	0.498
	DistilBERT	0.500	0.500	0.495	0.497
Base Model	BERT	0.831	0.813	0.859	0.835
	DistilBERT	0.827	0.823	0.833	0.828
Advanced Model	BERT	0.829	0.814	0.854	0.833
	DistilBERT	0.826	0.808	0.855	0.831
Layer Freezing	BERT	0.832	0.844	0.816	0.830
	DistilBERT	0.814	0.819	0.806	0.812
Optuna Tuning	BERT	0.832	–	–	–
	DistilBERT	0.824	–	–	–

Table 9: Comprehensive performance comparison of BERT and DistilBERT across experimental configurations. Metrics shown are Accuracy (Acc.), Precision (Prec.), Recall (Rec.), and F1-score (F1). Dashes indicate unavailable metrics from the tuning stage.

3 Results and Overall Analysis

3.1 Experimental Summary

This report presents a comprehensive study comparing the performance of BERT and DistilBERT models across multiple training configurations. Beginning with a **dummy model**, which offered random predictions and served as a baseline, we progressively fine-tuned more complex architectures and hyperparameters to improve sentiment classification accuracy on tweet data.

The experimentation process followed a logical progression:

1. **Dummy Model:** Provided baseline metrics, confirming that random predictions yield accuracy near 50%, with no meaningful classification performance.
2. **Base Model:** Fine-tuned the pretrained models with standard parameters, resulting in significant performance gains—accuracy over 83% for BERT and 82% for DistilBERT.
3. **Advanced Model:** Used more specific HuggingFace model classes, with results showing slightly lower accuracy but greater consistency across experiments.
4. **Layer Freezing:** Freezing the first six layers showed minor gains for BERT but led to a drop in DistilBERT’s performance, ultimately deeming this approach suboptimal.
5. **Hyperparameter Tuning:** Leveraged the Optuna framework to identify optimal hyperparameters, achieving the highest validation accuracy in the process: 83.24% for BERT and 82.41% for DistilBERT.

Table 9 summarizes the validation performance of both models across all major experimentation stages. The improvements from dummy to tuned models clearly demonstrate the effectiveness of fine-tuning transformer models on domain-specific tasks such as sentiment analysis.

3.2 Overall Comparison and Limitations

Comparing this project with the previous two, we observe a steady progression in model complexity and performance. The first project, using TF-IDF and Logistic Regression, reached an accuracy of approximately **0.80**, outperforming the second project’s best result of **0.7730** using word embeddings and feed-forward neural networks (FNN). However, through more refined techniques in this third project—BERT and DistilBERT transformer-based architectures—we achieved an improved accuracy of **0.8324**, surpassing both prior efforts.

While we intended to conduct a more extensive hyperparameter analysis, including individual testing and visualizations, the large dataset size, model complexity, and time constraints limited us to a small number of automated tuning trials. Further experimentation remains a promising direction for future personal work to optimize performance even more effectively.

3.3 Conclusion

Among the tested configurations, **BERT consistently outperforms DistilBERT** in accuracy and robustness. However, DistilBERT still offers competitive results with reduced computational requirements, making it a viable choice for resource-constrained environments. Ultimately, transformer models prove to be highly effective tools for sentiment classification tasks, especially when fine-tuned with appropriate architectures and hyperparameters.

4 Bibliography

References

- [1] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition, 2025. Online manuscript released January 12, 2025.
- [2] Despina-Athanasia Pantazi. Deep learning in nlp - bert for sentiment classification. <https://www.kaggle.com/competitions/ai-2-dl-for-nlp-2025-homework-3>, 2025. Kaggle.
- [3] Despina-Athanasia Pantazi. Deep learning in nlp - distilbert for sentiment classification. <https://www.kaggle.com/competitions/ai-2-dl-for-nlp-2025-homework-3-distil-bert>, 2025. Kaggle.

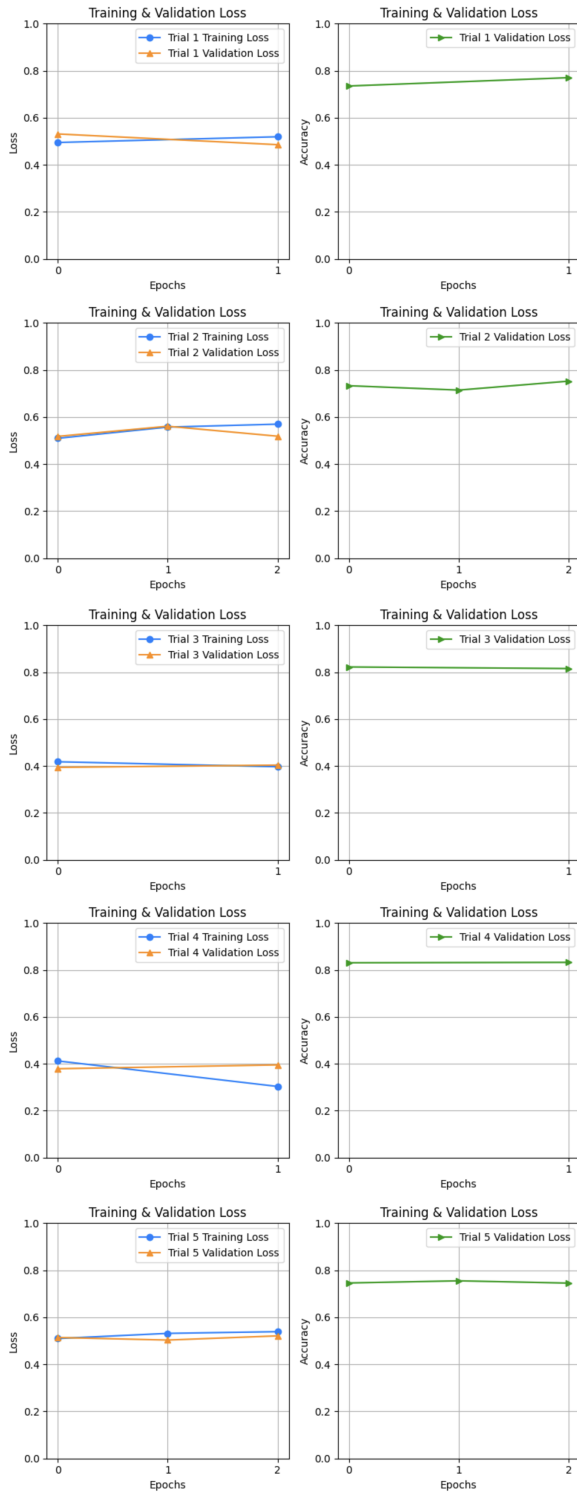


Figure 14: BERT model: Training and validation performance across all trials of Optuna Framework.

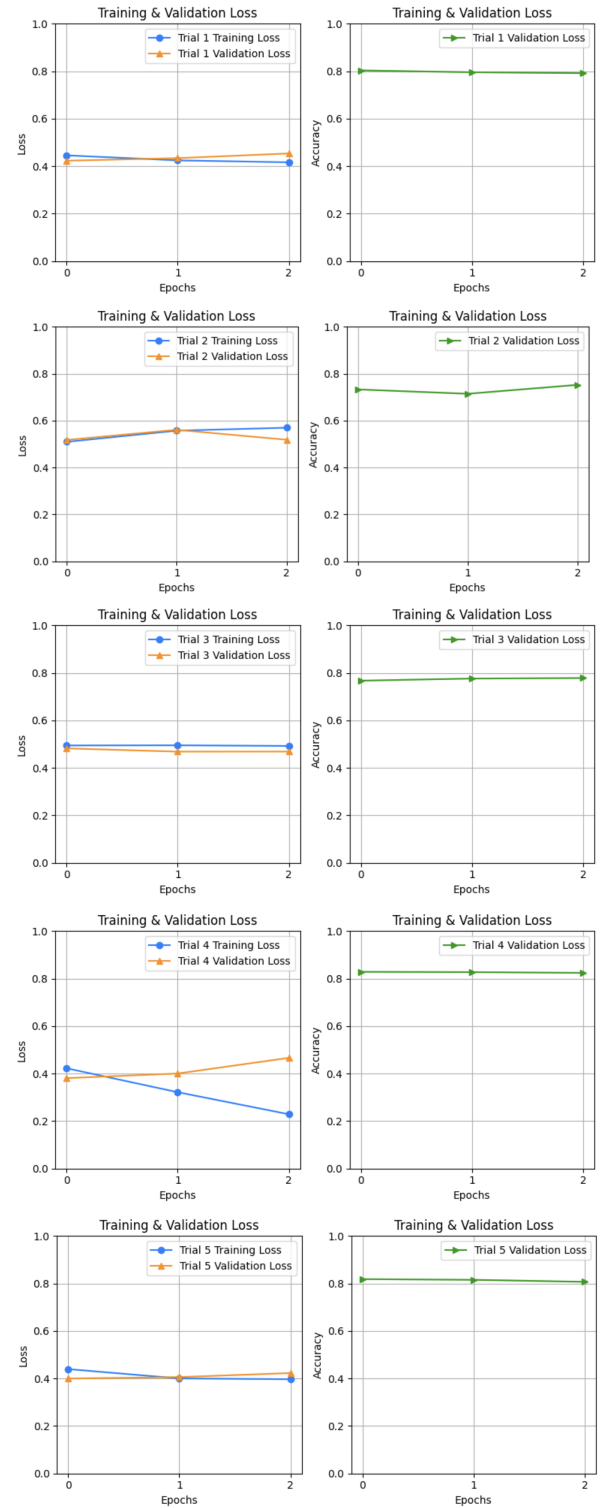


Figure 15: DistilBERT model: Training and validation performance across all trials of Optuna Framework.